

Datenbanken

Übung 7

Dozent: M. Sc. Burak Boyaci

Version: 12. Januar 2026

Wintersemester 25/26

1. Öffnen Sie pgAdmin 4 und verbinden Sie sich mit Ihrer Datenbank.
2. Schauen Sie sich bitte die Beispiele zu PL/pgSQL aus dem Skript noch einmal an und versuchen Sie die Beispiele nachzuvollziehen. Es ist sinnvoll, dass Sie die Beispiele selbst einmal in Ihrer Datenbank ausführen und die Ergebnisse nachvollziehen.

Hinweis: Die nachfolgenden Aufgaben beziehen sich alle auf das Datenmodell von FloppyFlight das aus den drei Tabellen Players, Sessions und Events besteht.

3. Welcher ist der kleinstmögliche anonyme Block, der mit PL/pgSQL dargestellt werden kann? (Hinweis: der Block muss nichts aus- bzw. zurückgeben).

Lösungsvorschlag:

```
1  do $$
2  begin
3  end $$;
```

4. Schreiben Sie bitte einen anonymen PL/pgSQL Block, der folgendes tut: Es soll auf der Konsole der Hinweis: *'Das ist der Kurs Datenbanken'* ausgegeben werden.

Ergebnis:

Data Output	Messages	Notifications
-------------	----------	---------------

NOTICE: Das ist der Kurs Datenbanken
DO

Query returned successfully in 658 msec.

Lösungsvorschlag:

```
1  do $$
2  declare v_text text := 'Das ist der Kurs Datenbanken';
3  begin
4  raise notice '%', v_text;
5  end $$;
```

5. Wir wollen die Anzahl der Spieler in der Tabelle *Players* herausfinden und diese Anzahl auf der Konsole ausgeben (printen).

Data Output Messages Notifications

INFO: Die Anzahl der FloppyFlight-Spieler: 44
DO

Query returned successfully in 49 msec.

Lösungsvorschlag:

```
1  do $$
2  declare
3  v_count integer;
4  begin
5  select count(*)
6  from floppy.players into v_count;
7  raise info ' Die Anzahl der FloppyFlight-Spieler beträgt: %', v_count;
8  end $$;
```

6. Schreiben Sie bitte einen anonymen PL/pgSQL Block, der folgendes tut: Es soll die *player_id* des *Players* 'name2' aus der Tabelle *Players* selektiert und anschließend der Variable *v_player* übergeben werden. Auf der Konsole sollen der Text 'Die ID des Spielers 'name2' lautet: ' und die ID des *Players* angezeigt werden.

Ergebnis:

Data Output Messages Notifications

INFO: Die ID des Spielers name2 lautet: 2299bbf2-d338-4281-a43b-9917e9d919a2
DO

Query returned successfully in 64 msec.

Lösungsvorschlag

```
1  do $$
2  declare v_player text;
3  begin
4  select player_id
5  from floppy.players into v_player
6  where player_name = 'name2';
7  raise info 'Die ID des Spielers name2 lautet: %', v_player;
8  end $$;
```

7. Gegeben sei folgender anonymer PL/pgSQL Block:

```
1  do $$
2  declare
3  v_number int := 1;
4  v_number2 v_number%type := 5;
5  begin
6  v_number = v_number + v_number2;
7  raise info 'Die Zahl betraegt: %', v_number2;
8  end $$;
```

Welche Aussage können wir über den Datentyp der Variable `v_number2` treffen?

Lösungsvorschlag:

Die Variable `v_number2` übernimmt dynamisch denselben Datentyp wie die Variable `v_number`, die ein `int` ist, `v_number2` ist daher auch vom Datentyp `int`.

8. Welche Aussage können wir in diesem Fall über den Datentyp der Variable `v_number2` treffen?

```
10 do $$
11 declare
12 v_number int := 1;
13 v_number2 classic_cars.customers.customernumber%type := 5;
14 begin
15 v_number = v_number + v_number2;
16 raise info 'Die Zahl betraegt: %', v_number2;
17 end $$;
```

Lösungsvorschlag:

Die Variable `v_number2` übernimmt dynamisch denselben Datentyp wie die Spalte `customernumber` aus der Tabelle `customers`, die ein `text` ist, `v_number2` ist daher auch vom Datentyp `text`.

9. Welches Ergebnis folgender anonymer PL/pgSQL Block?

```
1  do $$
2  begin
3  update floppy.players
4  set consent_given = 'False'
5  where player_name = 'name2';
6  end $$;
```

Lösungsvorschlag:

Der anonyme Block updatet die Tabelle Players im Schema Floppy. Für den Spieler *name2* wird die Spalte *consent_given* in den Wert ‚False‘ geändert.

10. Gegeben sei folgender Anfang eines anonymen PL/pgSQL Blocks:

```
12  do $$
13  declare
14  v_x int := 10;
15  v_y int := 2;
16
17
18
19
```

Schreiben Sie bitte den Block zu Ende und berücksichtigen Sie dabei folgendes: Es soll eine Fallunterscheidung stattfinden. Für den Fall $v_y > v_x$ soll der Text: *v_y ist groesser als v_x* ausgegeben werden. Für den Fall $v_y < v_x$ soll der Text: *v_y ist kleiner als v_x* ausgegeben werden. Für den Fall $v_y = v_x$ soll der Text: *v_y ist gleich als v_x* ausgegeben werden. Führen Sie Ihren anonymen PL/pgSQL Block anschließend bitte aus, um Ihr Ergebnis zu prüfen.

Lösungsvorschlag:

```
1  do $$
2  declare
3  v_x int := 10;
4  v_y int := 2;
5  begin
6  if v_y > v_x then
7  raise notice 'v_y ist groeßer als v_x';
8  elseif v_y = v_x then
9  raise notice 'v_y ist gleich v_x';
10 elseif v_y < v_x then
11 raise notice 'v_y ist kleiner als v_x';
12 end if;
13 end $$;
```

11. Schreiben Sie bitte einen anonymen PL/pgSQL Block, der folgendes tut: Definieren Sie bitte eine E-Mail Variable `v_email`, die Sie mit dem Wert `'test@htw-berlin.de'` initialisieren. Definieren Sie nun bitte in der Execution folgende Fallunterscheidung: Falls der Wert der Variable `v_email` dem Wert `'test@htw-berlin.de'` entspricht, soll die Information 'Die E-Mail Adresse lautet: `v_email`' ausgegeben werden. Dabei soll `v_email` dem konkreten Wert der Variable `v_email` entsprechen. In allen anderen Fällen soll die Fehlermeldung: 'Unbekannte E-Mail Adresse' ausgegeben werden.

Lösungsvorschlag:

```
1  do $$
2  declare v_email text := 'test@htw-berlin.de';
3  begin
4  if v_email = 'test@htw-berlin.de' then
5  raise info 'Die E-Mail Adresse lautet: %', v_email;
6  else
7  raise exception 'Unbekannte E-Mail Adresse';
8  end if;
9  end $$;
```

12. Verändern Sie nun bitte den Wert der Variable `v_email` zu: `'test@htw-berlin.com'` und führen Sie den Block noch einmal aus. Was stellen Sie fest?

Lösungsvorschlag:

- ➔ Der Fall Unbekannte E-Mail Adresse tritt ein und es wird entsprechende Meldung auf der Konsole ausgegeben.

Data Output Messages Notifications

ERROR: Unbekannte E-Mail Adresse
CONTEXT: PL/pgSQL function inline_code_block line 7 at RAISE

SQL state: P0001

13. Erweitern Sie die Tabelle `Sessions` bitte um eine weitere Spalte `'Test'`, der Sie bitte den Datentyp `text` vergeben.

Lösungsvorschlag:

```
1  alter table floppy.sessions
2  add column Test text;
```

14. Schreiben Sie bitte einen anonymen PL/pgSQL Block, der folgendes tut: Die leere Spalte 'Test' in der Tabelle *Sessions* soll mit dem Text 'Dies ist ein Test' befüllt werden. Selektieren Sie bitte anschließend die Tabelle *Sessions* und prüfen Sie, ob der Text angepasst wurde.

Lösungsvorschlag:

```
1  do $$
2  begin
3  update floppy.sessions set test = 'Dies ist ein Test';
4  END $$;
5
6  SELECT * FROM floppy.sessions;
```

15. Im Folgenden sehen Sie die gespeicherte Prozedur *update_final_score*. Welche Unterschiede fallen Ihnen im Vergleich mit dem anonymen PL/pgSQL Block auf?

```
16  -- Prozedur anlegen
17  CREATE OR REPLACE PROCEDURE floppy.update_final_score(
18      p_session_id text,
19      p_new_score int
20  )
21  LANGUAGE plpgsql
22  AS $$
23  BEGIN
24      UPDATE floppy.sessions
25      SET final_score = p_new_score
26      WHERE session_id = p_session_id;
27
28      RAISE NOTICE 'Score aktualisiert: session=% neue Punktzahl=%', p_session_id, p_new_score;
29  END
30  $$;
```

Lösungsvorschlag:

1. Da eine Prozedur in der Datenbank-Engine gespeichert wird (im Gegensatz zu einem anonymen Block), muss Sie mit CREATE definiert werden.
2. Eine Prozedur erlaubt die Definition von Parametern in Klammern nach der Definition der Prozedur (in diesem Beispiel die beiden IN Parameter *p_session_id* und *p_new_score*).
3. In der Prozedur muss eine Sprache definiert werden, in der die Engine laufen soll (hier plpgsql).